

# EnergyScore — LIVRABLE1

## Livrable 1 — Plan de sécurité et budget

**Projet Cloud — Master IA, 2iE**

**Groupe :** KONE M'PIE AIMAN KHALIS RAKINE & Andriampitahiana Mamy Herin'Ny Avo RABESIAKA

**Domaine :** Énergie — Prédiction de charge énergétique de bâtiments

**Date :** Juin 2026

---

### 1. Présentation du projet

#### Problème résolu

Les architectes et bureaux d'études énergétiques manquent d'outils rapides pour estimer la performance thermique d'un bâtiment en phase de conception. EnergyScore permet de prédire la **charge de chauffage** (kWh/m<sup>2</sup>) d'un bâtiment à partir de ses caractéristiques architecturales, sans simulation thermique complète.

#### Utilisateurs visés

- Architectes et maîtres d'œuvre
- Bureaux d'études énergétiques
- Promoteurs immobiliers souhaitant anticiper les performances BBC/RE2020

#### Modèle ML

- **Type :** Régression supervisée (prédiction continue)
- **Algorithme :** Gradient Boosting Regressor (scikit-learn 1.9.0)
- **Dataset :** UCI Energy Efficiency — 768 bâtiments simulés, 8 features architecturales
- **Features :** compacité relative, surface totale, surface des murs, surface du toit, hauteur, orientation, surface de vitrage, distribution du vitrage
- **Cible :** Charge de chauffage Y1 (kWh/m<sup>2</sup>)
- **Performance :** RMSE  $\sim$  0.5 kWh/m<sup>2</sup>, R<sup>2</sup>  $\sim$  0.998 sur le jeu de test

#### Plateforme cloud retenue

**AWS ECS Fargate** — retenu car utilisé en séance et disponible via les crédits académiques. Les images Docker sont hébergées sur Docker Hub (CI/CD) puis sur AWS ECR (production).

---

### 2. Schéma d'architecture prévu

```
Internet
  |
  v
[Navigateur]
  | HTTPS (port 443)
  v
[Frontend React + nginx - port 5173]
```

```

| /api/* -> proxy vers backend
v
[Backend FastAPI – port 8000]
+— POST /api/predict ← JWT requis, rate-limité (20 req/min)
+— GET /api/health ← public (ALB healthcheck)
+— POST /api/auth/google ← vérification token Google
+— GET /api/admin/* ← JWT admin requis
|
+— [PostgreSQL 16] ← interne (subnet privé)
+— [S3] ← stockage model.pkl (subnet privé via endpoint)

```

**Frontières public / interne : - Public :** frontend (nginx), endpoint /api/health, /api/auth/google -  
**Authentifié :** /api/predict (JWT Google requis) - **Admin uniquement :** /api/admin/\* - **Interne :**  
 PostgreSQL, S3 (jamais exposés directement)

### 3. Analyse de risques

| #  | Risque                                                           | Catégorie    | P | I | Score | Mesure                                         |
|----|------------------------------------------------------------------|--------------|---|---|-------|------------------------------------------------|
| 1  | Clé API AWS exposée dans un commit git                           | Secrets      | 4 | 5 | 20    | .gitignore + trufflehog en CI bloquant         |
| 2  | Données de consommation bâtiments clients accessibles sans auth  | Données      | 3 | 5 | 15    | Endpoint /predict protégé par JWT Google       |
| 3  | Bucket S3 contenant model.pkl exposé en public                   | Config cloud | 3 | 4 | 12    | Block Public Access S3 + politique IAM stricte |
| 4  | Injection via les champs numériques du formulaire                | Application  | 2 | 4 | 8     | Validation Pydantic (types stricts, min/max)   |
| 5  | Modèle de prédiction biaisé (zones climatiques non représentées) | IA / modèle  | 3 | 3 | 9     | Audit par segment en CI, seuil RMSE bloquant   |
| 6  | Instance EC2 de test laissée allumée hors séance                 | Coûts        | 4 | 4 | 16    | Alerte budget AWS à 80 % + arrêt programmé     |
| 7  | Flood de requêtes sur /api/predict (model stealing)              | Réseau       | 3 | 3 | 9     | Rate limiting slowapi (20 req/min/IP)          |
| 8  | Conservation des historiques de prédictions > 2 ans              | Conformité   | 3 | 5 | 15    | Politique de rétention BDD 12 mois max         |
| 9  | Image Docker backend avec CVE critique non corrigée              | Config cloud | 3 | 4 | 12    | Scan Trivy en CI, base image mise à jour       |
| 10 | Accès admin sans MFA à la console AWS                            | Secrets      | 2 | 5 | 10    | MFA obligatoire + alertes CloudTrail           |

**Catégories couvertes :** [x] Secrets [x] Données [x] Config cloud [x] Application [x] IA/modèle [x] Coûts [x] Réseau [x] Conformité

**Top 3 prioritaires (score  $\geq 15$ ) :** 1. **Score 20** — Fuite de clés AWS -> trufflehog CI + GitHub Secrets 2. **Score 16** — Coût EC2 dérapant -> alerte budget 80 % 3. **Score 15** — Données sans auth + rétention RGPD -> JWT + politique 12 mois

## 4. Gestion des accès (IAM)

### Rôles applicatifs

| Rôle  | Accès                                                 | Authentification                                 |
|-------|-------------------------------------------------------|--------------------------------------------------|
| user  | POST /api/predict,<br>lecture historique<br>personnel | JWT Google (token Google -> JWT applicatif)      |
| admin | Dashboard stats,<br>historique complet,<br>export     | JWT Google + email dans ADMIN_EMAILS             |
| ci    | Push images ECR,<br>lecture S3 modèle                 | Rôle IAM OIDC GitHub Actions (pas de clé en dur) |

### Rôles AWS IAM

| Rôle                 | Permissions                                       | Usage                                     |
|----------------------|---------------------------------------------------|-------------------------------------------|
| ecsTaskExecutionRole | ECR pull, CloudWatch Logs                         | ECS Fargate (pull image + logs)           |
| ecsTaskAppRole       | S3 GetObject sur s3://bucket/model.pkl uniquement | Lecture du modèle ML au démarrage         |
| github-actions-oidc  | ECR push, ECS UpdateService                       | CI/CD GitHub Actions (OIDC, sans clé AWS) |

### Stockage des secrets

| Secret            | Emplacement                  | Jamais dans        |
|-------------------|------------------------------|--------------------|
| GOOGLE_CLIENT_ID  | GitHub Secrets + ECS env var | Code, .env.example |
| JWT_SECRET        | GitHub Secrets + ECS env var | Code, logs         |
| DOCKERHUB_TOKEN   | GitHub Secrets               | Code               |
| Clés AWS CI       | Rôle OIDC (pas de clé)       | Partout            |
| POSTGRES_PASSWORD | ECS Secrets Manager          | Code, logs         |

## 5. Estimation des coûts

**Hypothèses :** 100 utilisateurs/mois, 5 prédictions/utilisateur/jour, modèle 6.6 MB, BDD < 1 GB, 0 GPU nécessaire.

### Option A — AWS ECS Fargate (référence cours)

| Poste             | Service                        | Détail                   | Coût mensuel  |
|-------------------|--------------------------------|--------------------------|---------------|
| Compute backend   | ECS Fargate 0.25 vCPU / 0.5 GB | ~730h/mois               | 7 \$          |
| Compute frontend  | ECS Fargate 0.25 vCPU / 0.5 GB | ~730h/mois               | 7 \$          |
| Base de données   | RDS PostgreSQL db.t3.micro     | 730h + 20 GB             | 15 \$         |
| Stockage modèle   | S3 Standard                    | 6.6 MB                   | < 0.01 \$     |
| Load Balancer     | ALB                            | 730h + LCU               | 18 \$         |
| Transfert données | —                              | 100 users × 5 req × 1 KB | < 1 \$        |
| CloudWatch Logs   | —                              | ~500 MB logs/mois        | 0.50 \$       |
| <b>Total AWS</b>  |                                |                          | <b>~48 \$</b> |

## Option B — Fly.io (alternative économique)

| Poste               | Service               | Coût mensuel |
|---------------------|-----------------------|--------------|
| Backend             | shared-cpu-1x, 256 MB | 3 \$         |
| Frontend            | shared-cpu-1x, 256 MB | 3 \$         |
| PostgreSQL          | Fly Postgres 1 GB     | 3 \$         |
| Stockage modèle     | Volume Fly 1 GB       | 0.15 \$      |
| <b>Total Fly.io</b> |                       | <b>~9 \$</b> |

**Choix retenu pour le cours :** AWS ECS Fargate (crédits académiques disponibles). Budget alerte configuré à 40 \$ (80 % du seuil).

## 6. Plan de mise en œuvre

Checklist des mesures de sécurité à appliquer en ECUE1 (Livrable 2) :

- ☒ .gitignore excluant .env, \*.pem, client\_secret\*.json
- ☒ .env.example sans aucune valeur réelle
- ☒ Scan secrets : **trufflehog** en CI bloquant (GitHub Actions)
- ☒ Audit dépendances Python : **pip-audit** en CI bloquant
- ☒ Scan image Docker : **Trivy** (informatif HIGH, bloquant CRITICAL)
- ☒ Authentification API : **JWT Google** sur /api/predict
- ☒ Rate limiting : **slowapi** 20 req/min/IP sur /api/predict
- ☒ Validation entrées : **Pydantic** (types, min/max sur les 8 features)
- ☒ Application déployée sur AWS ECS Fargate — ALB : http://microscore-alb-843872004.eu-west-1.elb.amazonaws.com
- ☒ MFA activé sur le compte AWS root et IAM admin
- ☒ Rôles IAM ECS configurés (ecsTaskExecutionRole, ecsTaskAppRole)
- ☐ HTTPS en production (certificat ALB / Let's Encrypt)
- ☐ Alerte budget AWS à 80 % (40 \$)
- ☐ Politique de rétention BDD 12 mois (script CRON de purge)